

Estructuras de Datos y Algoritmos

Tema 6: Listas

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

¿Otra lista?

Ya usamos `list` como una secuencia con acceso por índice. Ahora estudiaremos otra forma de representar esa secuencia y cómo cambia el costo de sus operaciones.

- El **TDA lista** describe qué podemos hacer con la secuencia.
- La **representación** determina cómo se almacenan y conectan sus elementos.

Tanto una `list` como una lista enlazada pueden crecer y modificarse. La diferencia no es que una sea estática y la otra dinámica.

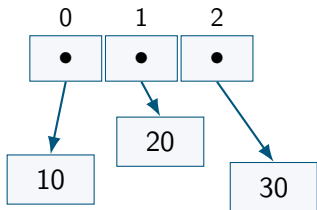
Pregunta

¿Qué operaciones resultan más convenientes con cada representación?

Representaciones

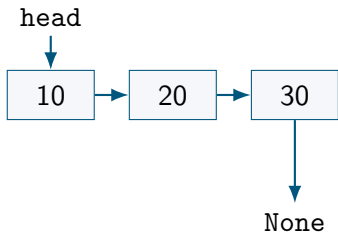
La misma secuencia: 10, 20, 30.

`list`: **arreglo dinámico de referencias**



En Python, las **referencias** ocupan posiciones contiguas.

Lista enlazada: nodos y referencias



El orden lo determinan los enlaces, no la cercanía física de los nodos.

Definición: TDA lista

Una lista es una secuencia finita de elementos en la que importa la posición. Permite valores repetidos y no exige que estén ordenados por valor. Sus operaciones se describen sin fijar una representación en memoria.

Relación con arreglos: ambos describen elementos por su posición. El TDA lista no exige acceso directo: puede implementarse mediante un arreglo dinámico o mediante nodos enlazados.

Consultar un elemento, recorrer la secuencia, insertar y remover son operaciones del TDA. También podemos crear la lista vacía y consultar su longitud.

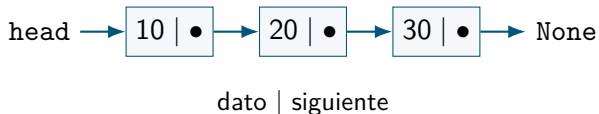
Para $L = (10, 20, 10)$, no es lo mismo indicar:

- una **posición**: $i = 2$ identifica el tercer elemento;
- un **valor**: buscar 10 requiere precisar cuál coincidencia;
- una **referencia a un nodo**: identifica un objeto concreto de una representación enlazada.

Lista enlazada

Definición: lista simplemente enlazada

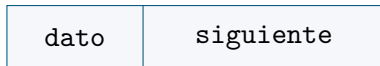
Una estructura lineal formada por nodos. Cada nodo almacena un dato y una referencia al siguiente nodo de la secuencia.



- head referencia el primer nodo.
- None indica ausencia de sucesor; **no es otro nodo**.
- Lista vacía: head = None.

Nodo en Python

```
class Nodo:  
    def __init__(self, dato, siguiente=None):  
        self.dato = dato  
        self.siguiente = siguiente
```



- dato: el valor almacenado.
- siguiente: una referencia a otro Nodo, o None.

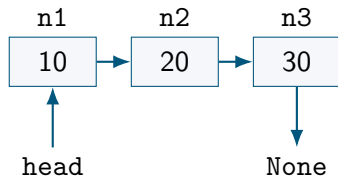
Construir una lista

```
n1 = Nodo(10)
n2 = Nodo(20)
n3 = Nodo(30)

n1.siguiente = n2
n2.siguiente = n3
head = n1
```

El constructor dejó `n3.siguiente` en `None`.

Una lista vacía basta `head = None`.



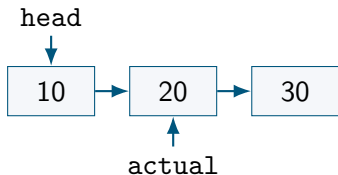
Crear los nodos, conectarlos y conservar una referencia al primero.

Referencias

Ambas trazas parten de $10 \rightarrow 20 \rightarrow 30$.

Mover una referencia

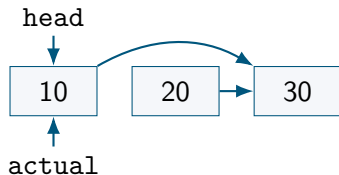
```
actual = head  
actual = actual.siguiente
```



Cambia el nodo al que se refiere actual; la cadena queda igual.

Cambiar un enlace

```
actual = head  
actual.siguiente = n3
```



Ahora desde head se alcanza $10 \rightarrow 30$. El nodo 20 queda fuera la cadena.

Recorrido

```
def recorrer(head):  
    actual = head  
    while actual is not None:  
        print(actual.dato)  
        actual = actual.siguiente
```

Traza de actual

Nodo con dato	Se imprime
10	10
20	20
30	30
None	termina

Cada avance sigue un enlace. En una lista finita sin ciclos se llega a None; si está vacía, no se entra al ciclo.

$$n \text{ nodos visitados} \implies T(n) \in \Theta(n).$$

Obtener por posición

```
def obtener(head, i):  
    if i < 0:  
        raise IndexError("índice negativo")  
    actual = head  
    posicion = 0  
    while actual is not None and posicion < i:  
        actual = actual.siguiente  
        posicion += 1  
    if actual is None:  
        raise IndexError("índice fuera de rango")  
    return actual.dato
```

i es un entero. Las posiciones comienzan en cero.

Para $10 \rightarrow 20 \rightarrow 30$:

- `obtener(head, 0)`
devuelve 10.
- `obtener(head, 2)`
devuelve 30.
- $i = -1$ o $i = 3$:
`IndexError`.

A diferencia de `list`, aquí no aceptamos índices negativos.

$$0 \leq i < n: \quad T(i) \in \Theta(i + 1)$$

peor caso: $\Theta(n)$.

Buscar un valor

```
def buscar(head, x):  
    actual = head  
    while actual is not None and  
        actual.dato != x:  
        actual = actual.siguiente  
    return actual
```

Devuelve el **primer nodo** cuyo dato coincide con x, o None si no existe.

Para $10 \rightarrow 20 \rightarrow 10$:

- buscar 10 devuelve el nodo de head;
- buscar 99 devuelve None.

obtener devuelve un **dato**; buscar, una **referencia a un nodo**. Esa referencia permite modificar enlaces después.

$T_{\text{buscar}}(n) \in \Theta(n)$ en el peor caso.

Insertar al inicio

```
def insertar_inicio(head, x):  
    nuevo = Nodo(x)  
    nuevo.siguiente = head  
    return nuevo
```

```
head = insertar_inicio(head, 10)
```



En ambos estados, 30 termina en None.

Agregar un nuevo nodo requiere un numero finito de operaciones que no depende del tamaño de la lista.

$$T(n) \in \Theta(1).$$

Eliminar al inicio

```
def eliminar_inicio(head):  
    if head is None:  
        raise ValueError("lista vacia")  
    eliminado = head.dato  
    head = head.siguiente  
    return head, eliminado
```

```
head, eliminado = eliminar_inicio(head)
```

Antes	Nueva cabeza	Dato retirado
10 → 20 → 30	nodo 20	10
10 → None	None	10

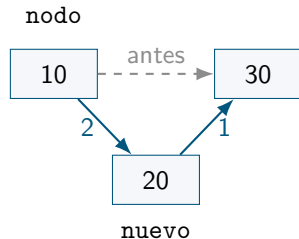
Eliminar un nodo requiere un numero finito de operaciones que no depende del tamaño de la lista.

$$T(n) \in \Theta(1)$$

Insertar después

Ya tenemos la referencia al nodo 10 y queremos insertar 20 después de él.

```
def insertar_despues(nodo, x):  
    if nodo is None:  
        raise ValueError("nodo no puede ser None")  
    nuevo = Nodo(x)  
    nuevo.siguiente = nodo.siguiente  
    nodo.siguiente = nuevo
```



1. El nuevo nodo conserva el enlace al resto de la cadena.
2. El nodo anterior se enlaza al nuevo.

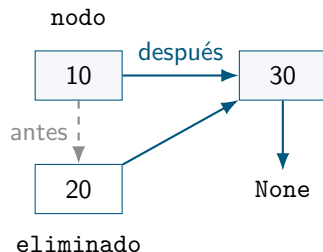
Insertar un nodo requiere un numero finito de operaciones que no depende del tamaño de la lista.

$$T(n) \in \Theta(1)$$

Eliminar después

Ya tenemos la referencia al nodo 10 y queremos retirar su siguiente, el nodo 20.

```
def eliminar_despues(nodo):  
    if nodo is None or nodo.siguiente is None:  
        raise ValueError("no hay nodo que eliminar")  
    eliminado = nodo.siguiente  
    nodo.siguiente = eliminado.siguiente  
    return eliminado.dato
```



Desenlazar el nodo no implica destruirlo: otra referencia podría conservarlo.

$$T(n) \in \Theta(1)$$

Localizar y modificar

costo total = localizar + modificar enlaces.

Situación	Localizar	Modificar
Insertar después de un nodo conocido	$\Theta(1)$	$\Theta(1)$
Insertar después del primer valor x	$\Theta(n)$	$\Theta(1)$
Eliminar después de un predecesor conocido	$\Theta(1)$	$\Theta(1)$
Eliminar el primer valor x , sin su predecesor	$\Theta(n)$	$\Theta(1)$

Los costos de búsqueda son de **peor caso**. Mientras buscamos un nodo que queremos retirar, podemos conservar también su predecesor.

Resultado

Cambiar pocos enlaces cuesta tiempo constante; encontrar dónde cambiarlos puede requerir recorrer toda la lista.

Lista como objeto

```
class ListaEnlazada:
    def __init__(self):
        self._head = None

    def esta_vacia(self):
        return self._head is None

    def insertar_inicio(self, x):
        self._head = Nodo(x, self._head)
```

```
lista = ListaEnlazada()
lista.insertar_inicio(20)
lista.insertar_inicio(10)
print(lista.esta_vacia())
# False
```

La cabeza queda guardada dentro del objeto.

La función anterior requería `head = insertar_inicio(head, x)`. Este método actualiza `self._head` y no devuelve una nueva cabeza.

El objeto reúne la representación y las operaciones del TDA. Aquí solo mantenemos `_head`, sin `tail` ni contador.

Peor caso, salvo (*): costo amortizado. Simple con solo head.

Operación	list (CPython)	Simple, solo head
Obtener por índice	$\Theta(1)$	$\Theta(n)$
Buscar un valor	$\Theta(n)$	$\Theta(n)$
Consultar longitud	$\Theta(1)$	$\Theta(n)$
Insertar al inicio	$\Theta(n)$	$\Theta(1)$
Eliminar al inicio	$\Theta(n)$	$\Theta(1)$
Insertar al final	$\Theta(1)$ (*)	$\Theta(n)$
Eliminar al final	$\Theta(1)$ (*)	$\Theta(n)$
Insertar en un lugar conocido	$\Theta(n)$	$\Theta(1)$
Eliminar en un lugar conocido	$\Theta(n)$	$\Theta(1)$

Lugar conocido: **índice** en list; **nodo anterior** en la simple (predecesor al eliminar).

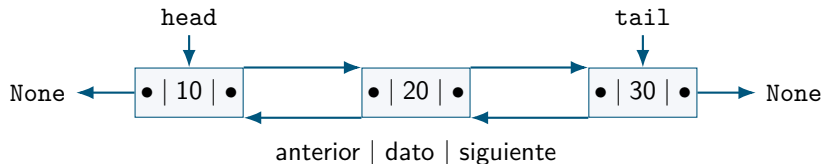
Con tail, insertar al final pasa a $\Theta(1)$. Con contador, consultar longitud pasa a $\Theta(1)$.

(*) Promedio por operación en una secuencia; redimensionar puede costar $\Theta(n)$. Suponemos datos y comparaciones de costo constante.

Doble enlace

Definición: lista doblemente enlazada

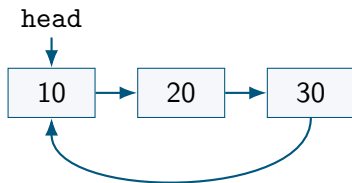
Cada nodo almacena un dato y dos referencias: una al nodo anterior y otra al siguiente.



- Podemos avanzar y retroceder.
- Desde un nodo conocido también accedemos a su predecesor.
- A cambio: una referencia adicional por nodo y más enlaces por actualizar.

Lista circular

En una lista circular no vacía, el último nodo vuelve a enlazar al primero.



Si hay un único nodo, su siguiente es él mismo.

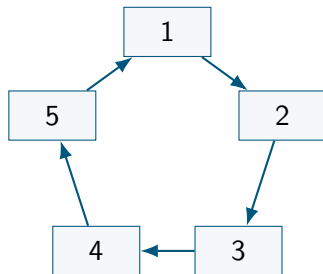
Circularidad y doble enlace son independientes: también puede haber una lista **doblemente enlazada circular**.

```
if head is not None:
    actual = head
    while True:
        print(actual.dato)
        actual = actual.siguiente
        if actual is head:
            break
```

Una vuelta termina al regresar a la cabeza.

Problema de Josefo

Contamos k participantes por ronda, incluyendo al inicial, y eliminamos al que recibe el número k . Aquí hay cinco participantes y $k = 2$: se elimina a cada segundo participante hasta que queda uno.



- 1 Comenzamos en el participante 1: cuenta como uno.
- 2 El siguiente cuenta como dos y se retira.
- 3 Reiniciamos la cuenta en el siguiente al eliminado.

Cada participante es un nodo. Contar significa avanzar por enlaces; retirar significa cambiar el enlace de su predecesor.

Inicio	Se elimina	Quedan
1	2	1, 3, 4, 5
3	4	1, 3, 5
5	1	3, 5
3	5	3

Después de eliminar, la cuenta reinicia en su sucesor.

Con esta estructura tenemos $n - 1$ eliminaciones con trabajo constante cada una, $\Theta(n)$. Construir el círculo cuesta $\Theta(n)$.

Basta una circular simple

Al inicio, `actual` apunta a `head` y `anterior` al último nodo.

Antes de avanzar, guardamos `actual` en `anterior`. Así conservamos el predecesor sin un enlace hacia atrás.

Al eliminar, enlazamos `anterior` con el sucesor de `actual`: $\Theta(1)$. Ese sucesor pasa a ser `actual`; `anterior` no cambia.

Elegir representación

Representación	Conviene cuando necesitamos	Desventajas
<code>list</code>	Acceso por índice y crecimiento al final.	Insertar o eliminar al inicio o en medio desplaza referencias: peor caso $\Theta(n)$.
Simple	Avanzar; insertar o eliminar después de un nodo conocido.	Acceso secuencial, sin retroceso. Eliminar exige predecesor (salvo al inicio).
Doble	Avanzar y retroceder; insertar antes o después de un nodo conocido, o eliminarlo.	Acceso secuencial; referencia extra por nodo y actualización en ambos sentidos.
Circular	Recorrer ciclos; insertar o eliminar conservando el círculo.	Otro criterio de paro (no hay <code>None</code>); mantener el cierre al modificar extremos.

En Python, `collections.deque` utiliza internamente una lista doblemente enlazada de bloques, no circular. Permite insertar y eliminar eficientemente en ambos extremos, pero no acceder directamente a sus nodos ni modificar sus enlaces. El paquete externo `l1ist` ofrece listas simples (`s1list`) y dobles (`d1list`) con acceso a sus nodos. Python no incluye en su biblioteca estándar una clase de lista enlazada circular.